Natural Language Processing

Group Assignment 1

Combined Report

Word Segmentation and POS Tagging · Dependency Parsing

Spelling Correction · Live Integrated Editor

Group Members

Name	Roll No.	Email
Shivam Singh	12342020	shivamsgh@iitbhilai.ac.in
Ayush Khelwal	12340430	ayushkl@iitbhilai.ac.in
Keshav Mishra	12341140	keshavm@iitbhilai.ac.in
Rishi Kharya	12341790	rishik@iitbhilai.ac.in
Akshat Gupta	12340160	akshatg@iitbhilai.ac.in

Indian Institute of Technology Bhilai

September 8, 2026

Contents

Overview

Assignment brief: `Group Assignment 1.pdf`. This document combines the four per-question reports (`Question-N/REPORT_QN.md`) into a single report. Each section below is reproduced in full from its source report; nothing has been condensed or altered. See each question's own folder for setup instructions, code, and raw evidence (logs, JSON results, sample-run transcripts) backing the numbers reported here.

Team

Name	Roll No.	Email
Shivam Singh	12342020	shivamsgh@iitbhilai.ac.in
Ayush Khelwal	12340430	ayushkl@iitbhilai.ac.in
Keshav Mishra	12341140	keshavm@iitbhilai.ac.in
Rishi Kharya	12341790	rishik@iitbhilai.ac.in
Akshat Gupta	12340160	akshatg@iitbhilai.ac.in

Contributions

Each member led one question end to end: implementation, training and evaluation, and the report.

Shivam Singh built Question 1, the word segmentation and POS tagging pipeline for English and Spanish, including training and evaluation and the report.

Ayush Khelwal built Question 2, the arc-standard dependency parser: CoNLL-U parsing, the oracle simulator, feature extraction, classifier training, LAS evaluation, and the report.

Keshav Mishra built Question 3, the spelling corrector: corpus and language models, both candidate-generation methods, non-word and real-word correction, evaluation and the Speed Demon benchmark, the interactive CLI, and the report.

Rishi Kharya built the first working version of Question 4: the Streamlit editor scaffolding, wiring in the Q1 decoder and Q3 corrector, and the PCFG parser.

Akshat Gupta took Question 4 the rest of the way (the shared n-gram grammar models, the scoring and method-selection pipeline, the Speed Demon benchmark, the experiment sweep, the Streamlit Cloud deployment, and the report), reorganized the top-level repo, and ran a final audit across all four questions that fixed a broken data-setup step, a missing report, an under-tuned threshold, and a bug where Q4 could silently overwrite Q1's trained model.

Question 1: Word Segmentation and POS Tagging

Word segmentation and POS tagging on unspaced text: English (Brown) vs Spanish (UD Spanish-GSD).

Every number below comes from one full run of `.venv/bin/python scripts/run_q1.py` — transcript in `models/q1_run_full.log`, machine-readable results in `models/q1_results.json`. Test samples are 400 sentences per language, drawn with `seed=42` from each corpus's own test split. All tuning was done on dev; test was consulted once.

Setup

	English (Brown)	**Spanish (UD Spanish-GSD)**
Train sentences	45,459	14,186
Train tokens	804,554	330,548
Word types	41,974	39,748
Type/token ratio	**0.052**	**0.120**
Mean token length	4.72 chars	4.80 chars
Mean sentence length	17.7 tokens	23.3 tokens
Dev perplexity (trigram word LM)	1,088.6	2,233.0
OOV rate on the test sample	2.44%	6.66%
Tagset size	11	16
Morphology-aware tagset size	11 (see §2)	78

The type/token ratio is the most useful number in the table: Spanish reaches almost the same vocabulary size as English from **a quarter of the tokens**. Inflection splits each lemma across many surface forms, so every form is seen fewer times. Mean token length is essentially identical (4.72 vs 4.80 chars), which rules out "Spanish words are longer" as an explanation for anything below.

Headline results

Measure	**English**	**Spanish**
Segmentation, greedy longest-match (baseline)	66.71%	50.20%
Segmentation, trigram DP + beam	**95.61%**	**91.59%**
Tagging on gold segmentation, most-frequent-tag (baseline)	93.64%	88.72%
Tagging on gold segmentation, trigram HMM	**96.19%**	**93.75%**
End to end, greedy + most-frequent (baseline)	68.68%	53.21%
End to end, DP segment then HMM tag	**92.62%**	**86.75%**
Agreement reproduced (gold words)	n/a	96.46% (2,625 pairs)

Segmentation is scored by token F1 (a predicted token counts only if its exact character span is a gold span); tagging by accuracy; end to end by accuracy over gold tokens, where a token whose span was never recovered counts as wrong.

1. Where did English and Spanish differ most?

In segmentation, and above all in the OOV tail — not in tagging.

Comparison	English — Spanish
Segmentation, exact sentences	**+27.75 pp** (66.50 vs 38.75)
Greedy segmentation baseline	**+16.51 pp** (66.71 vs 50.20)
End-to-end baseline	+15.47 pp
Tagging, **unknown words only**	**+9.78 pp** (86.31 vs 76.53)
End to end, full pipeline	+5.81 pp
DP segmentation (token F1)	+3.96 pp
Tagging given gold segmentation	+2.44 pp
Tagging, **known words only**	+1.45 pp (96.43 vs 94.98)

Three things follow.

The difficulty is concentrated in segmentation. Once the words are correctly separated, Spanish tagging trails English by only 2.44 pp — and by just 1.45 pp on known words. The end-to-end gap of 5.81 pp is therefore mostly inherited from the segmenter rather than generated by the tagger.

Sparse statistics, not grammar, drive the gap. Spanish's larger relative vocabulary means every form is seen fewer times: perplexity is double English's (2,233 vs 1,089) *despite* Spanish having the smaller LM vocabulary, and the OOV rate is nearly triple (6.66% vs 2.44%). The single widest gap after segmentation is unknown-word tagging (+9.78 pp), which is exactly the place where a model has no counts to fall back on.

The characteristic Spanish failure is a frequent function word available as the prefix of a rarer content word. From the brief's own sample string:

```
elcielodespejadoesazul
-> el/DET cielo/NOUN de/ADP spejado/NOUN es/AUX azul/ADJ
```

`despejado` is split into the very frequent preposition `de` plus a non-word `spejado`. English produces this class of error far less often simply because its frequent function words collide with fewer content-word prefixes.

Caveat: Spanish is also being scored on a 16-tag set against English's 11, so the tagging comparison is not perfectly like-for-like and slightly flatters English.

2. Did agreement-aware tagging help, or add noise?

It helped in Spanish. In English the question cannot be asked at all.

English is a null result about the corpus, not the language. The morphology-aware tags are built from UD `FEATS`, which Brown does not carry. With no gender or number to attach, the refined tagset is *identical* to the plain one (11 tags either way), so the two taggers are the same model. Nothing is learned here about English; the annotation needed to ask the question is absent.

Spanish, scored naively, looks worse — and that comparison is invalid. On its own label set the morphology-aware HMM scores 91.72% against the plain tagger's 93.75%. But these are not the same task: the morph tagger chooses among 78 labels rather than 16. A lower number on a harder task is not evidence of noise.

Scored on the same decision, it helps. Projecting the morphology-aware tagger's output back to coarse tags puts both models on the identical 16-way decision:

Model	Coarse-tag accuracy
Plain trigram HMM	93.75%
Morphology-aware HMM, projected to coarse	**94.23%** (+0.48 pp)
Plain most-frequent-tag	88.72%
Morphology-aware most-frequent-tag, projected	89.74% (+1.02 pp)

So refining the tagset **added no noise**: both models improved on the identical decision. But note honestly that the context-free baseline gained *more* than the HMM did, so this table on its own does not establish that the gain comes from agreement modelling — splitting the tagset changes which tag wins a per-word argmax, and that alone can help.

The direct evidence that agreement was learned is stronger. Reading it straight out of the transition table, in the context (matching determiner, noun):

Context	P(agreeing ADJ)	P(clashing ADJ)	Ratio
`DET-Masc-Sg NOUN-Masc-Sg __`	0.0848	0.0047	**18x**
`DET-Fem-Sg NOUN-Fem-Sg __`	0.0866	0.0065	**13x**
`DET-Fem-Pl NOUN-Fem-Pl __`	0.1042	0.0085	**12x**

An adjective that does not mark gender at all (`grande -> ADJ-Sg`) is counted as underspecified, not as a clash; counting it as a clash understates the effect roughly tenfold.

And it delivers something the plain tagger cannot express. Of 2,625 adjacent gold-agreeing pairs, agreement was reproduced in **96.46%**:

Context	Pairs	Agreement reproduced
DET+NOUN	1,163	97.68%
NOUN+ADJ	281	97.15%
ADJ+NOUN	156	98.72%
DET+ADJ	116	99.14%
NOUN+NOUN	24	70.83%
DET+PROPN	14	71.43%

The brief's examples come out fully correct, including the number flip propagating across all four agreeing words:

```
lacasarojaesgrande      -> la/DET-Fem-Sg casa/NOUN-Fem-Sg roja/ADJ-Fem-Sg es/AUX-Sg
    ↪ grande/ADJ-Sg
lascasasrojassongrandes -> las/DET-Fem-Pl casas/NOUN-Fem-Pl rojas/ADJ-Fem-Pl son/AUX-
    ↪ Pl grandes/ADJ-Pl
```

Two honest observations. First, "agreement reproduced" and "agreement carries the *correct* values" are identical in every context: there are **zero** pairs where the model propagates a consistently wrong gender, because the emission model pins the value from the word form. Second, end to end the same figure falls to **86.06%**, because 305 of those pairs contain a word the segmenter never recovered — agreement cannot survive a word that was never found.

3. Segmentation-induced vs genuine tagging errors

An error is **segmentation-induced** if the gold token's character span was never recovered (so no tag decision was made for it) and **genuine** if the span was correct and the tag wrong.

System	Errors	Seg-induced	Genuine	% from segmentation	Tag acc. given correct span
English					
greedy + most-frequent (baseline)	2,152	1,861	291	**86.5%**	94.19%
DP segment -> most-frequent tag	661	273	388	41.3%	94.12%
DP segment -> HMM tag	511	273	238	**53.4%**	96.39%
Spanish					
greedy + most-frequent (baseline)	4,583	4,162	421	**90.8%**	92.53%
DP segment -> most-frequent tag	1,576	831	745	52.7%	91.69%
DP segment -> HMM tag	1,298	831	467	**64.0%**	94.79%

With a weak segmenter, segmentation is essentially the only error source (86.5% English, 90.8% Spanish). The baseline's tagging is not the problem: given a correctly segmented token it tags at 94.19% / 92.53%. Nearly all of its end-to-end failure is inherited.

Improving the tagger raises the segmentation share, which is the point. Going from the most-frequent-tag baseline to the HMM on the same segmenter cuts genuine errors sharply (388 -> 238 English, 745 -> 467 Spanish) and leaves segmentation-induced errors untouched by construction, so their share rises from 41.3% to 53.4% and from 52.7% to 64.0%. That number going *up* is the tagger working.

Spanish remains segmentation-dominated (64.0% vs 53.4%). Conditional tagging accuracy is high and similar in both languages (96.39% vs 94.79% given a correct span), which confirms that what separates the two end to end is where the words go, not what they are called.

The two classes look completely different in practice:

Segmentation-induced (tagger never saw the word)	Genuine (word right, tag wrong)
`unfailing` -> `un` + `failing`	`eran` VERB -> AUX
`dallasbased` -> `dallas` + `based`	`que` PRON -> SCONJ
`burocracias medicas` -> `burocraciasmedicas`	`bolsillo` NOUN -> PROPN

Nothing a tagger could do about the left column. The right column is genuine ambiguity — and the top confusions confirm it: English mixes VERB/NOUN (35+24) and PRT/ADP (17+16); Spanish mixes PROPN/NOUN (75+50, unsurprising once case is normalised away) and PRON/DET (42).

4. How much better than the simple baselines?

Task	Baseline	Model	Absolute gain	Error reduction
English				
Segmentation (token F1)	66.71%	95.61%	+28.91 pp	**86.8%**
Segmentation (exact sentences)	16.25%	66.50%	+50.25 pp	60.0%
Tagging (gold segmentation)	93.64%	96.19%	+2.55 pp	40.1%
End to end	68.68%	92.62%	+23.94 pp	**76.4%**
Spanish				
Segmentation (token F1)	50.20%	91.59%	+41.39 pp	**83.1%**
Segmentation (exact sentences)	5.75%	38.75%	+33.00 pp	35.0%

Task	Baseline	Model	Absolute gain	Error reduction
Tagging (gold segmentation)	88.72%	93.75%	+5.03 pp	44.6%
End to end	53.21%	86.75%	+33.54 pp	**71.7%**

Both baselines were given every advantage: greedy longest-match gets the *full* training vocabulary including hapax words the language model itself discards, and most-frequent-tag falls back to the majority tag among rare training words rather than a blanket NOUN.

Segmentation is where the model pays for itself — it removes 83-87% of the baseline's errors. Greedy longest-match has no way to reconsider: one wrong early bite corrupts every boundary after it, which is why it segments only 16.25% of English and 5.75% of Spanish sentences perfectly, against 66.50% and 38.75% for the DP decoder.

Tagging is where the baseline is already strong. Most-frequent-tag reaches 93.64% / 88.72%, because most word types are simply unambiguous. The HMM's +2.55 / +5.03 pp is a real 40-45% error reduction, but it operates on a small residue of genuinely ambiguous tokens. Reporting only the end-to-end figure would badly overstate what the tagging model contributes.

The cost is real. Greedy runs in 0.04 ms/sentence against 8 ms for the deployed beam decoder in English (0.06 vs 25 ms in Spanish) — roughly 200x and 400x slower respectively, for those 29-41 points of F1. Beam search is what makes that affordable: checked against brute-force exact DP on a 40-sentence sample, it comes within 0.3 points of exact accuracy in English (96.82% beam vs 96.53% exact, at the selected width of 4) and matches it exactly in Spanish (92.91% vs 92.91%, at the selected width of 8), while running about 40-50x faster than exact DP (8 ms vs 391 ms in English, 25 ms vs 947 ms in Spanish).

5. Known failures and caveats

Stated plainly, since two are on the brief's own sample strings.

- `brown` -> **NOUN, not ADJ.** In `thequickbrownfoxjumpsoverthelazydog` every word is segmented correctly, but `brown` is tagged NOUN where the brief expects an adjective. In the Brown corpus the form is predominantly a proper noun (it is the corpus's namesake), so both the baseline and the HMM prefer NOUN.
- `despejado` -> `de` + `spejado`. Discussed in §1.
- `pueden` -> **AUX, where the brief writes VERB.** A tagset convention, not an error: UD annotates modal `poder` as AUX, and the model was trained on UD.
- **English morphology was never tested.** Brown carries no `FEATS`, so the agreement analysis is Spanish-only. A UD English treebank would be needed to complete that half of the comparison.
- **Sample size.** All test figures are 400 sentences per language; differences below roughly half a point should not be read as real.

Question 2: Transition-Based Dependency Parser

1. Objective

The goal of this project is to build a simple, data-driven dependency parser from scratch. The parser uses a transition-based approach with the Arc-Standard transition system. A scikit-learn classifier, trained on a treebank, is used to predict parsing transitions.

2. Dataset

The **Universal Dependencies English-EWT** (English Web Treebank) corpus is used: `en_ewt-ud-train.conllu` for training (12,544 sentences) and `en_ewt-ud-dev.conllu` for evaluation (2,001 sentences).

The dataset was obtained by cloning the official repository: `https://github.com/UniversalDependencies/UD_English-EWT.git`

3. CoNLL-U Data Representation

Each sentence in the CoNLL-U file is parsed into a `Sentence` object containing a list of `Token` objects. Each `Token` stores:

Field	Description
`id`	Token index (1-based; 0 for ROOT)
`form`	Word form / token text
`upos`	Universal POS tag
`head`	Gold-standard head token ID (0 = root)
`deprel`	Dependency relation label

A virtual ROOT token (id=0, POS="ROOT") is added at index 0 of every sentence. Multi-word tokens and empty nodes are skipped during parsing as they are not relevant for dependency tree construction.

4. Arc-Standard Transition System

The parser uses the **Arc-Standard** transition system with three transitions:

1. **SHIFT**: Move the first word from the buffer to the top of the stack.
2. **LEFT-ARC(label)**: The word at the top of the stack becomes the head of the second word on the stack. The second word is then popped from the stack. An arc `(top` $\rightarrow$ `second, label)` is created.
3. **RIGHT-ARC(label)**: The second word on the stack becomes the head of the word at the top of the stack. The top word is then popped from the stack. An arc `(second` $\rightarrow$ `top, label)` is created.

Initial configuration: Stack = [ROOT], Buffer = [word_1, word_2, ..., word_n], Arcs = $\emptyset$

Terminal condition: Buffer is empty and stack contains only ROOT.

5. Oracle Simulation

The oracle simulates the parsing process on gold-standard trees to generate training data. At each configuration, it determines the correct transition:

1. **LEFT-ARC(label)**: Applied when the second item on the stack has its gold head equal to

the top of the stack, AND all dependents of the second item have already been collected.

2. **RIGHT-ARC(label)**: Applied when the top of the stack has its gold head equal to the second item, AND all dependents of the top item have already been collected.

3. **SHIFT**: Applied as the default when neither arc condition is met.

The dependent-completeness check is what makes this work: a word can only be removed from the stack after all of its children in the gold tree have been attached to it. That's what guarantees the oracle produces transitions that reconstruct the exact gold dependency tree.

The oracle generated **409,156 training instances** across **88 unique transition labels** from the 12,544 training sentences.

6. Feature Extraction

For each parser configuration, exactly **4 features** are extracted:

#	Feature	Description
1	Stack top POS	POS tag of the word on top of the stack
2	Stack second POS	POS tag of the second word on the stack
3	Buffer first POS	POS tag of the first word in the buffer
4	Buffer second POS	POS tag of the second word in the buffer

When a position is unavailable (e.g., stack has fewer than 2 elements), the special sentinel value `NONE` is used. Features are encoded as categorical variables using scikit-learn's `DictVectorizer`, which creates one-hot encoded representations. This resulted in a feature matrix of shape **(409,156 × 73)**.

7. Classifier

Model: Logistic Regression (scikit-learn `LogisticRegression`)

Configuration: `max_iter=1000` (enough iterations to reach convergence), `solver='lbfgs'` (works well for multinomial classification), `C=1.0` (the default regularization strength, left untouched), and `n_jobs=-1` so training uses all available CPU cores.

Why Logistic Regression: it trains fast even on a large dataset like this one (409K instances, done in about 80 seconds), and it handles the 88 transition labels naturally through multinomial softmax rather than needing a one-vs-rest wrapper. The one-hot encoded POS features are sparse, which logistic regression is efficient with, and the model stays interpretable, which matters for an assignment where the point is understanding the parser rather than squeezing out accuracy. It also outputs class probabilities instead of just a single label, which the parser leans on to fall back to the next most likely transition when the top prediction turns out to be invalid.

Training accuracy: 80.33%

8. Parser Implementation

The parser takes a sentence (words + POS tags) as input and produces dependency arcs:

1. Initialize: stack = [ROOT], buffer = [1..n], arcs = []

2. At each step:

   - Extract the 4 POS features from the current configuration

- Use the trained classifier to predict transition probabilities
- Select the highest-probability **valid** transition
- Apply the transition to update the configuration

3. Continue until the buffer is empty and the stack has at most 1 element

Validity checking is what keeps the parser from crashing: SHIFT is only valid if the buffer is non-empty, LEFT-ARC needs the stack to have at least 2 elements with the second one not being ROOT, and RIGHT-ARC just needs the stack to have at least 2 elements.

If no predicted transition is valid, a fallback mechanism applies:

1. SHIFT if the buffer has elements
2. RIGHT-ARC(dep) if the stack has ≥ 2 elements
3. Otherwise, parsing terminates

A safety limit of `4n + 10` steps prevents infinite loops.

9. LAS Calculation

Labeled Attachment Score (LAS) measures the percentage of tokens for which **both**:

1. The predicted head is correct (matches gold head)
2. The predicted dependency label is correct (matches gold label)

Formula: `LAS = (correct tokens / total tokens)` $\times$ `100`

Only non-ROOT tokens are evaluated. The ROOT token (id=0) is excluded from the count.

10. Final LAS Score

Metric	Value
Sentences processed	2,001
Total tokens evaluated	25,148
Correct tokens	14,254
Labeled Attachment Score	**56.68%**
Evaluation time	5.8s

This LAS of **56.68%** is consistent with expectations for a simple transition-based parser using only 4 POS-tag features and Logistic Regression. More sophisticated parsers use additional features (word forms, lemmas, morphological features, contextual embeddings) and more powerful models (neural networks) to achieve higher accuracy.

11. Example Outputs

Sentence 1: "The cat sat on the mat."

ID	Word	Head	Head Word	Relation
1	The	2	cat	det
2	cat	0	ROOT	root
3	sat	2	cat	acl
4	on	6	mat	case

ID	Word	Head	Head Word	Relation
5	the	6	mat	det
6	mat	3	sat	obj
7	.	2	cat	punct

Sentence 2: "She eats a green salad."

ID	Word	Head	Head Word	Relation
1	She	2	eats	nsubj
2	eats	0	ROOT	root
3	a	5	salad	det
4	green	5	salad	amod
5	salad	2	eats	obj
6	.	2	eats	punct

Sentence 3: "I saw the man with a telescope."

ID	Word	Head	Head Word	Relation
1	I	2	saw	nsubj
2	saw	0	ROOT	root
3	the	4	man	det
4	man	2	saw	obj
5	with	7	telescope	case
6	a	7	telescope	det
7	telescope	4	man	nmod
8	.	2	saw	punct

12. Design Choices

Logistic Regression was picked over SVM or Random Forest mainly for speed with this many classes, plus it gives natural probability outputs and plays well with sparse one-hot features. Feature encoding uses `DictVectorizer`, which gives clean one-hot encoding of the categorical POS features without having to hand-roll label encoding. For transition selection, the parser doesn't just take the top prediction; it walks down the ranked list of transitions by probability and applies the first one that's actually valid, which cuts down parse failures a lot compared to always trusting the top-1 prediction. The oracle's dependent-completeness check is there to keep the training data correct, since a word can only be reduced once all of its children have been attached. And the virtual ROOT token exists just to simplify the transition system, so there's always a root anchor sitting on the stack instead of having to special-case an empty stack.

13. Limitations

The parser only uses 4 POS-tag features, which caps how context-sensitive its decisions can be; adding word form features, dependency relation features from children already attached, or a bit more buffer lookahead would likely help. Related to that, it has no lexical knowledge at all, it only sees POS tags, so it can't pick up on word-specific patterns. Parsing is also greedy: the model commits to a transition at each step with no search, so beam search or some form of global optimization would probably raise accuracy further. The oracle itself is static rather than dynamic, meaning the model has no way to recover from its own prediction errors during training the way dynamic-oracle techniques allow. Finally, Arc-Standard can only produce projective trees, so non-projective dependencies in the gold data fall back to the generic fallback mechanism and aren't

always parsed correctly.

14. Conclusion

This project builds a working transition-based dependency parser on the Arc-Standard system, trained on the Universal Dependencies English-EWT treebank and evaluated on the dev set, where it reaches a Labeled Attachment Score of **56.68%**. That's a modest number next to state-of-the-art parsers, but it still covers the core ideas of data-driven dependency parsing: generating training data through an oracle, extracting features from parser configurations, predicting transitions with a classifier, and evaluating the result with a standard metric. The code is split into clear modules and handles its edge cases without crashing.

Question 3: Spelling Corrector

This is the technical report referenced by `README.md`. The README is the grading map (which file/command covers which marked part); this document covers design choices, the numbers behind them, and an honest account of where the corrector does and doesn't work.

All numbers below come from a fresh run of `evaluation.py`, `benchmark.py`, and `cli.py` in this tree (`NLTK_DATA` pointed at the submitted `nltk_data/`, Brown reporting 57,340 sentences; see README §7 for why this count is environment-dependent).

1. Part 1: Corpus and Models

`corpus_models.py` builds three artifacts from the Brown Corpus, cleaned to lowercased alphabetic tokens (`utils.clean_sentence`):

Artifact	Value
Vocabulary size	40,234 unique words
Total unigram tokens	981,716
Unique bigram types	388,815
Smoothing constant `k`	1.0 (add-k / Laplace-style)

Bigrams are counted **within sentences only**: `zip(sent, sent[1:])` never crosses a sentence boundary, so `bigram_counts` has no spurious cross-sentence pairs. `bigram_log_prob` returns

```
P(w2 | w1) = (count(w1, w2) + k) / (count(w1) + k * |V|)
```

as a natural log, so unseen bigrams and unseen `w1` degrade gracefully to a smoothed floor instead of zero. Sanity check: `P(the | of) = 0.127` (`the` is Brown's single most frequent word, so a high probability after a common preposition is expected).

`k = 1.0` was not tuned against a held-out metric. The assignment does not require it, and Part 1 is scored on correct add-k implementation, not on an optimal `k`. It is exposed as a parameter on every function that needs it (`bigram_log_prob`, `SpellingCorrector`) specifically so it *could* be retuned later without retraining the counts.

2. Part 2: Candidate Generation

Two independent methods generate the same target set (all vocabulary words within edit distance 1), by different mechanisms:

- **Method A** (`edit_distance_1_candidates`): brute-force. For a word of length n, generates every deletion, insertion, replacement, and transposition string (`O(n · |Σ|)` strings for a 26-letter alphabet), then filters against the vocabulary.
- **Method B** (`build_symdel_index` + `symdel_candidates`): Symmetric Delete. At startup, every vocabulary word has all of its one-character deletions computed once and indexed (`{deletion: [originals]}`). At query time, only the misspelled word's own one-character deletions are computed and looked up in that index. Insertions, replacements, and transpositions are recovered as a side effect of the *deletion-only* index (a word reachable from the query by inserting a character is exactly a word whose own deletion matches the query), verified with an explicit edit-distance check before being accepted.

Both were empirically checked against each other (300 randomly corrupted words) and against an independent brute-force Damerau-Levenshtein-$\leq$1 checker: **identical candidate sets in every case, zero mismatches.** This is also *why* Method A and Method B report identical accuracy

throughout this report: they retrieve the same candidate universe by construction; the difference (see §4) is purely how fast they do it.

3. Part 3: Correction Logic

Non-word correction (`correct_nonword`): for a word absent from the vocabulary, generate candidates (Method A, B, or both) and return the one with the highest raw unigram frequency, breaking ties alphabetically for determinism.

Real-word correction (`correct_realword`): for a word that *is* in the vocabulary, compare

```
score(word) = log P(word | previous) + log P(next | word)
```

for the original word against every edit-distance-1 candidate (including the original in its own candidate set), and switch only if the best candidate beats the original by more than `real_word_threshold` nats.

3.1 Choosing `real_word_threshold`

The assignment does not fix a threshold ("if a candidate phrase has a *significantly* higher probability...", "significantly" is left to the implementer). The first working version used `2.0`, chosen without evidence. Sweeping it against the Part 4 real-word test set (5,734 cases) tells a different story:

Threshold (nats)	Real-word accuracy	False-positive rate*
2.0 (original default)	62.31%	1.93%
1.5	67.60%	n/a
1.1 (**chosen**)	74.75%	4.23%
1.0	76.07%	4.58%
0.5	81.25%	6.71%
0.1	82.09%	8.96%

* False-positive rate = share of already-correct in-vocabulary words that get "corrected" anyway, measured by running `correct_realword` on every in-vocabulary word position across an independent, uncorrupted 2,000-sentence Brown sample (34,790 word positions), i.e. how often the corrector breaks something that wasn't broken.

The obvious move is "pick the highest accuracy" (0.1, at 82%). That would be wrong: as the threshold drops, the corrector isn't getting smarter, it's firing more often on tiny, noise-level probability gaps, and the false-positive rate climbs in lockstep (8.96% at 0.1 vs 1.93% at 2.0). **`1.1` was chosen** as the point past which additional accuracy comes at a false-positive cost that grows faster than the accuracy gain (1.1→1.0 buys +1.3pp accuracy for +0.35pp more false positives; 1.1→0.5 buys +6.5pp accuracy for +2.5pp more false positives), and, more concretely, for the reason in §3.2 below.

3.2 Case study: why "meat" is left unchanged, on purpose

The assignment's own real-word examples are "I would like to **sea** the world" and "Please **meat** me at the station." At `threshold=1.1`:

```
> Original:  I would like to sea the world.
Corrected: I would like to **see** the world.
```

```
> Original:  Please meat me at the station.
Corrected: Please meat me at the station.
```

The second one *looks* like a miss, but it's deliberate. Ranked candidates for "meat" in this exact context:

Candidate	Score	Margin over "meat"
beat	−20.11	**+1.10**
meet	−20.52	+0.69
meaty / mea / mead	−21.21	≈ 0.00
(meat, original)	−21.21	n/a

"beat" outranks "meet" in this local bigram context (Brown has stronger support for phrases like "...to beat me..." than "...to meet me...") **regardless of the threshold**: the ranking order doesn't change, only whether *anything* fires does. At `threshold=1.0` the corrector does fire here, but it says **"Please beat me at the station"**, a confident, wrong answer. At `1.1` (margin needed: >1.1, actual best margin: 1.10) it just barely stays under the bar and leaves "meat" alone.

Between "silently wrong" (unchanged) and "confidently wrong" (→ "beat"), unchanged was the better failure mode, so `1.1` was chosen over the higher-scoring `1.0` specifically to keep this example on the safe side of the line. This is a genuine limitation of scoring with only immediate bigram context (`P(word|prev)` and `P(next|word)`) rather than the whole sentence: nothing in "Please _____ me at the station" locally disambiguates "meet" from "beat", since both are transitive verbs a person can do to another person. Fixing this would need a wider context window (trigram+ or a syntactic cue), which is out of scope for the assignment's bigram-based design.

3.3 What this trade-off looks like in practice

Two full CLI transcripts are in §6. The first (the assignment's own four example sentences) shows the corrector working as intended. The second, with different self-chosen sentences, shows the honest cost of `1.1`:

```
> Original:  The qwuick brown fox jumped over the lazy dog.
Corrected: The **quick** brown fox jumped over the **lady** dog.
```

"qwuick"→"quick" is a correct non-word fix. "lazy"→"lady" is a real-word **false positive**: "lazy" was already correct. Checking its margin, "lady" beats "lazy" by 1.94 nats in this context, just over the `1.1` bar. The same run also flips "we"→"he" (margin 1.84) and "wore"→"were" (margin 1.60); neither would have fired at the original `threshold=2.0`. One more, "bank"→"back" (margin 5.43), is large enough that it would have misfired even at the original, more conservative default, i.e. it is not something this threshold change introduced, but a pre-existing weakness of scoring "bank" only against its immediate neighbors in a corpus where "to the back" is a far more common phrase than "to the bank" is common in this context.

This is reported rather than hidden because it is the honest answer to "did context-aware real-word correction actually help, or add noise?" The answer is **both**: it roughly doubles the assignment's real-word test accuracy (62.31% → 74.75%, +12.4pp) while measurably increasing how often it touches text that didn't need touching (1.93% → 4.23% of already-correct words in casual text). A local-bigram-only design cannot fully separate "a rare but valid word choice" from "a real-word error"; it can only make the trade-off explicit and pick a defensible point on that curve, which is what §3.1 does.

4. Part 4: Evaluation and Speed Demon

4.1 Accuracy

Test set: 10% of Brown sentences (5,734 cases at this environment's corpus size; see README §7), one single-edit corruption per sentence, generating both a non-word and a real-word version of each case from the same underlying sentence and seed (`seed=42`, fully reproducible).

Method	Non-word accuracy	Real-word accuracy
Method A	4679 / 5734 = **81.60%**	4286 / 5734 = **74.75%**
Method B	4679 / 5734 = **81.60%**	4286 / 5734 = **74.75%**

(Identical across methods; see §2 for why.)

4.2 Error analysis by edit type

Breaking the same test set down by which single-edit operation produced the corruption (deletion / insertion / replacement / transposition) shows the errors are not uniform:

Non-word correction:

Edit type	Accuracy
Insertion	1650/1766 = **93.43%**
Replacement	1241/1521 = 81.59%
Transposition	1163/1446 = 80.43%
Deletion	625/1001 = **62.44%**

Real-word correction:

Edit type	Accuracy
Insertion	424/458 = **92.58%**
Replacement	1000/1198 = 83.47%
Transposition	308/379 = 81.27%
Deletion	2554/3699 = **69.05%**

Deletion is the hardest error type in both tasks, by a wide margin. Deleting a character from a word tends to produce a *shorter* string with disproportionately many valid deletion-neighbors, e.g. correcting "ho" (from "how") has **28 candidates** to rank among, versus a handful for a typical insertion/replacement corruption. The more candidates compete, the more likely unigram/bigram frequency picks a *plausible but wrong* one (observed failures: "into"→"int"→**"in"**, "two"→"tw"→**"to"**, "how"→"ho"→**"to"**). Insertion is the easiest, because removing the one inserted character usually recovers a much smaller, less ambiguous candidate set.

4.3 Speed Demon

Isolated non-word correction logic, batch of exactly 1,000 misspelled words, identical batch and order through both methods, model/index construction excluded from timing:

Run	Method A	Method B	Speedup
Run 1	0.1130 s	0.0108 s	≈ 10.43×
Run 2	0.0835 s	0.0089 s	≈ 9.40×

Why Method B is faster, specifically: Method A enumerates the *entire* edit-distance-1 string space for each query word: every deletion, insertion (×25 letters at every one of *n+1* positions), replacement (×25 letters at every position), and transposition, an $O(n \cdot |\Sigma|)$ string set that is regenerated from scratch, in full, for every single query, and then each generated string is checked against the vocabulary set. Method B only ever generates a word's n one-character **deletions** at query time (no insertion/replacement/transposition strings are ever materialized) and looks each one up in a precomputed hash index, so its query-time cost is $O(n)$ string generations plus

`O(n)` hash lookups, against Method A's `O(n · |Σ|)` generations plus set-membership checks. The ~9-10× ratio observed is consistent with the alphabet-size gap this removes (Method A generates roughly 25× more insertion/replacement strings alone, and adding transpositions widens the gap further); Method B recovers those same insertion/replacement/transposition matches for free as a side effect of the precomputed *deletion* index rather than generating them at query time.

5. Part 5: Live Interactive CLI

`cli.py` runs a continuous loop (`run_cli`) that reads a sentence, calls `corrector.correct_sentence`, and prints:

- `Corrected:`: the corrected sentence, with every changed word wrapped in `**asterisks**` in place (`render_highlighted`), per the assignment's highlighting requirement.
- `Changes:`: a redundant, more explicit `original -> corrected` summary line (kept in addition to inline highlighting, not instead of it).
- `Latency:`: wall-clock time of exactly the `correct_sentence` call, in milliseconds, measured with `time.perf_counter()` around nothing else (not the prints, not the input read; model/index construction happens once at startup in `create_corrector`, before the loop begins, so it never leaks into a per-sentence number).

Edge cases handled without crashing: blank input (prompts again rather than correcting nothing), stdin closing without an explicit `exit` (prints "End of input. Goodbye." and exits 0 rather than raising `EOFError`), and the exact string `exit` (prints "Goodbye." and exits).

6. Sample Runs

Run 1: the assignment's own four example sentences:

```
> Original:  I hav a good feeling about this.
Corrected: I **had** a good feeling about this.
Changes: hav -> had
Latency: 0.410 ms

> Original:  This is a test sentnce.
Corrected: This is a test **sentence.**
Changes: sentnce -> sentence
Latency: 0.307 ms

> Original:  I would like to sea the world.
Corrected: I would like to **see** the world.
Changes: sea -> see
Latency: 0.390 ms

> Original:  Please meat me at the station.
Corrected: Please meat me at the station.
Changes: none
Latency: 0.348 ms

> Goodbye.
```

Run 2: self-chosen sentences, mixing non-word and real-word errors:

```
> Original:  The qwuick brown fox jumped over the lazy dog.
Corrected: The **quick** brown fox jumped over the **lady** dog.
Changes: qwuick -> quick, lazy -> lady
Latency: 0.526 ms
```

```
> Original:  I recieved your mesage yesterday.
Corrected: I **received** your **message** yesterday.
Changes: recieved -> received, mesage -> message
Latency: 0.243 ms

> Original:  We went to the bank to withdraw money.
Corrected: **He** went to the **back** to withdraw money.
Changes: we -> he, bank -> back
Latency: 0.616 ms

> Original:  She wore a beutiful dress to the party.
Corrected: She **were** a **beautiful** dress to the party.
Changes: wore -> were, beutiful -> beautiful
Latency: 0.597 ms

> Goodbye.
```

Run 2 is included deliberately, not cherry-picked for a clean result; see §3.3 for what its false positives show about the real-word threshold trade-off.

7. Comparative Analysis Summary

- **Do Method A and Method B differ in what they find?** No, verified identical on 300 random cases plus the full 5,734-case test set (§2). They differ only in speed (§4.3), by roughly an order of magnitude, for the mechanistic reason given there.
- **Did real-word (context-aware) correction help, or add noise?** Both, and the two effects were measured separately (§3.1, §3.3): +12.4pp accuracy on genuine errors, +2.3pp false-positive rate on already-correct text, at the chosen threshold. Neither number alone tells the full story.
- **Which error type is hardest?** Deletion, for both non-word and real-word correction, because deleting a character produces a shorter string with more competing valid neighbors (§4.2).
- **How much faster is Method B, and why exactly?** ~9-10×, because it replaces Method A's full per-query enumeration of the edit-distance-1 string space with `O(n)` deletions looked up in a precomputed index (§4.3). Not because it checks fewer candidates, but because it never generates most of them.

8. Limitations

- Coverage is limited to the Brown Corpus vocabulary (40,234 words), a ~1960s corpus; some correct modern words will be treated as unknown, and some archaic Brown-only words rank as unexpectedly strong candidates.
- Contractions and hyphenated forms (e.g. "don't", "well-known") are not handled like normal alphabetic words, since vocabulary/cleaning filters on `str.isalpha()` and excludes tokens containing apostrophes or hyphens.
- Real-word correction depends on **local bigram context only** (previous word and next word). It cannot resolve cases like "meat"/"meet"/"beat" that need wider context or world knowledge to disambiguate (§3.2), and it has a measurable, quantified false-positive rate on correct text (§3.1, §3.3) that is an inherent property of the design, not a bug.
- Only edit-distance-1 corrections are generated (per the assignment scope); two-edit errors are out of scope for both methods.

Question 4: Live Integrated Editor

The live editor from Question 4 runs three sub-systems over one stream of text: the segmentation and POS decoder trained in Question 1, the spelling corrector built in Question 3, and the PCFG parser and n-gram grammar models added here. This report covers the shared language models, the end-of-passage analysis, the deployment and the benchmark, and it answers the comparison questions in the brief.

Live deployment: nlp-live-editor.streamlit.app

Every number below comes from a script in `scripts/`, and the raw output is kept in `reports/`. Nothing here was typed in by hand.

What	Command	Output
Add-k sweep	`venv/bin/python scripts/tune_lm_k.py`	table in section 1.3
PCFG floors	`venv/bin/python scripts/calibrate_pcfg.py`	`models/q4_pcfg_floors.json`
Sample runs	`venv/bin/python scripts/run_passage_demo.py --seed 1`	`reports/sample_run_seed1.txt`, `sample_run_seed5.txt`, `sample_run_seed7.txt`
Experiments	`venv/bin/python scripts/run_experiments.py`	`reports/experiments.txt`
Speed Demon	`venv/bin/python scripts/run_speed_demon.py`	`reports/speed_demon.txt`
Live app	`venv/bin/streamlit run app.py`	`reports/live_app.png`, `reports/live_app_analysis.png`

Which model comes from where

Nothing from Question 1 or Question 3 is retrained while Q4 runs. `q4/model_loader.py` loads all of it once and hands the same objects to every sub-system. If Q1's or Q3's own artifact is missing (e.g. Q4 is run before either has been trained), `model_loader.py` falls back to training a local copy for that run — but caches it under `Question-4/models/`, never back into `Question-1/models/` or `Question-3/.../models/`, so a fallback run can never silently overwrite the real artifact those questions' own scripts produce with a different-schema substitute. Run Q1's and Q3's own setup first (see the top-level README) so Q4 reuses the real trained models rather than this fallback.

Model	Trained in	Used for
Witten-Bell trigram LM	Q1, Brown train split of 45,459 sentences	scoring segmentation splits in the beam decoder
Trigram HMM tagger, 41,974 known word forms	Q1, same split	POS tags for the split words and for the parser
Vocabulary of 40,234 words, unigram and bigram counts, SymDel index	Q3, Brown	non-word and real-word spelling correction
Add-k bigram, 23,780 word vocabulary	Q4, Brown train split	sentence scoring in the end-of-passage table
Add-k trigram, same vocabulary	Q4, Brown train split	live window perplexity and sentence scoring
Pruned Penn Treebank PCFG, 10,603 productions	Q4, treebank sample	constituency parses

The beam decoder runs with the settings Q1 selected and persisted alongside its models, so Q4 does not re-choose them: maximum word length 19, beam width 4 (Q1's dev-set sweep found widths 4/8/16 tie to within 0.001 token F1 on English, so whichever ties first wins — see Question-1/REPORT_Q1.md), alpha and beta both 1.0.

The two Q4 n-gram models are the only ones trained here. They are cached in `models/q4_grammar_lms.pkl`, which is rebuilt on first run in about 13 seconds and left out of git because it is 31 MB.

1. Parameter choices

1.1 Merge probability p = 0.08

The value comes from Part 1 and it is kept. At p = 0.08 a passage of 100 words gets about eight dropped spaces, which is frequent enough that most sentences contain one without the passage turning into a stress test. The sweep in `reports/experiments.txt` shows what p buys:

p	segment alerts over 20 passages	alerts per 100 words
0.00	30	1.27
0.04	115	4.87
0.08	177	7.50
0.16	304	12.89

The p = 0 row is the useful one. No token was merged in that run, so all 30 alerts are false, which puts the segmentation false-alert rate at 1.3 per 100 words. They are almost all long words Q1 never saw, such as `fastidiousness` splitting into `fastidious` and `ness`. Raising p does not move that floor, it only adds real merges on top of it, so the share of alerts that are wrong falls as p rises.

1.2 Trigger interval N = 10

The grammar check reads the last N words every N words. Running it on clean Brown dev text gives a direct false-alert rate, since real published text should not be flagged at all:

N	windows	fired	perplexity only	real-word only	false-alert rate
5	400	64	52	11	16%
10	400	48	31	17	12%
20	400	80	6	73	20%

The two causes pull in opposite directions. Short windows are dominated by perplexity spikes, because five words are not enough context for the trigram and one rare word drags the whole window over the threshold. Long windows are dominated by the real-word check, because every extra word is another chance for some candidate to beat it on a bigram score. N = 10 sits at the minimum of the two effects. It also bounds how late an error is caught: a problem is seen at worst ten words after it is typed, which at two words per second is five seconds.

1.3 Add-k constant

Both grammar models are fitted on the Brown train split and scored on the dev split:

k	bigram dev perplexity	trigram dev perplexity
0.001	788.6	**1788.9**
0.01	**763.8**	2048.2
0.1	1172.5	3651.8
0.5	2049.3	6215.0
1.0	2738.4	7796.5

The two orders want different constants, so they each get their own: k = 0.01 for the bigram and

k = 0.001 for the trigram. The reason is sparsity. Add-k spreads k units of count over every word in the vocabulary for every context, and a trigram has far more contexts with tiny counts than a bigram does, so the same k drains much more mass away from the events that were actually observed. A single shared k = 0.01 would cost the trigram 14 percent in perplexity for no gain.

Both models are checked for normalisation at training time with `check_normalised`, because the decision rule compares scores across sentence lengths and a model that leaks probability mass would bias that comparison.

1.4 Perplexity threshold

Part 1 used a fixed threshold of 300, which was tuned against Q1's Witten-Bell model. The add-k trigram lives on a completely different scale, and a fixed 300 would fire on almost every window. The threshold is now measured instead. `q4/ngram_lm.py` scores 3,866 ten-word windows from held-out Brown text and takes the 95th percentile, which is 17,344. That puts roughly 5 percent of clean in-domain windows over the line by design. Measured again on a different slice of dev text in section 1.2 it comes out at 8 percent for perplexity alone, and 12 percent once the real-word check is counted as well.

2. Tagset reconciliation

Q1 tags with the 12 tag universal set and the PCFG is built over Penn Treebank tags, so `q4/tagset.py` maps each universal tag to the Penn tag it corresponds to most often, for example `NOUN` to `NN` and `VERB` to `VBD`. The map is many to one, which is where the loss comes from: `NN`, `NNS`, `NNP` and `NNPS` all arrive as `NN`.

Measured over 200 treebank sentences:

Measure	**Value**
Mapped tag equals the treebank tag	58.2%
Parse rate using the mapped tags	39.0%
Parse rate using the treebank's own tags	42.5%

So the mapping loses about 42 percent of tag identity but only 3.5 points of parse rate. That gap is explained by how the parser uses tags. A word the grammar has seen is parsed through its own lexical productions and the predicted tag is ignored, so the tag only matters for words the grammar does not know. Retagging Q1 at training time would have removed even that cost, but it would mean retraining Q1's tagger, which the brief rules out.

3. Method selection rule

Each finished sentence is scored three ways and one score is chosen:

1. Use the PCFG when the sentence parses, is at most 25 words, and its per-word parse log probability is at least the 2nd percentile of parses of real treebank sentences (-9.11). A parse below that is treated as an outlier and not trusted.
2. Otherwise use the trigram, if at least 80 percent of the sentence is in its vocabulary.
3. Otherwise use the bigram.

The verdict comes from the chosen score against floors measured the same way, on 2,000 held-out Brown sentences for the n-grams and 400 treebank sentences for the parser:

Model	10th percentile, borderline below	2nd percentile, ungrammatical below
PCFG per word	-7.81	-9.11
Bigram per word	-7.87	-8.81
Trigram per word	-8.70	-9.55

Two design points are worth stating. Scores are length-normalised, otherwise every long sentence would look ungrammatical next to a short one. And the 25 word cap on parsing is not arbitrary: CKY cost grows with the cube of the length, and treebank sentences beyond that length are rare enough that the pruned grammar has little to say about them.

The rule fires as intended. Over 128 sentences from 20 passages the parser was chosen 21 times, the trigram 101 times and the bigram 6 times.

4. Speed Demon benchmark

A batch of exactly 1,000 corrupted words is built with Question 3's generator, then run through the per-token layer and through the grammar trigger separately. Model loading, the SymDel index and the batch itself are all prepared before timing starts.

Layer	Corrupted batch	Ordinary words
Segmentation and spelling, per word	1.723 ms	0.785 ms
Grammar trigger, per window of 10	0.049 ms	0.577 ms
Grammar trigger, per word	0.005 ms	0.058 ms

On the corrupted batch the per-token layer costs about 1.72 ms per word more than the grammar layer. The ratio (≈351x) looks dramatic but it is measuring a worst case at both ends. Every word in that batch is out of vocabulary, which is the most expensive case for segmentation because the beam decoder has to run, and the cheapest case for the grammar check because the real-word pass skips unknown words without generating a single candidate. The control batch of ordinary vocabulary words is the fairer picture: the per-token layer drops to well under half because known short words skip the decoder entirely, and the grammar check climbs per window because it now has real words to generate candidates for. (Absolute numbers here are from a shared, loaded machine and will vary run to run — see the two consecutive runs of this same script in `reports/speed_demon.txt`'s history — but the ~2x gap between the corrupted and ordinary batches, and the two-orders-of-magnitude gap between the per-token and per-trigger layers, reproduce consistently.)

The conclusion is that no throttling is needed. Even at the worst case of ~1.7 ms per word, a typist at 120 words per minute leaves 500 ms between words, so the check uses well under 1 percent of the available time. The measured live figures agree: 0.19 ms mean per token over 20 passages, with a 95th percentile of 1.06 ms and a worst single token of 21.6 ms, which is one very long merged token that the beam decoder had to work through. Throttling the segmentation check to the grammar trigger would only delay the split of a merged token by up to ten words and would save nothing that matters.

5. Comparative analysis

5.1 Live alerts against the final verdict

Over 128 sentences from 20 passages:

Case	Sentences
Flagged live and at the end	18

Case	Sentences
Flagged live only	78
Flagged at the end only	3
Clean for both	29
Agreement	37%

The 78 sentences flagged live only are not a failure. A segmentation or spelling alert is a repair, so a sentence that had `thedormitoryand` split correctly is flagged live and then scores as ordinary English at the end, because by then it is ordinary English. The live layer reports what it fixed and the final layer reports what the text looks like after the fixes, so they should disagree on exactly those sentences. Run 1 in `reports/sample_run_seed1.txt` is the clearest case: 13 segmentation alerts and 5 grammar alerts, and all six sentences come out grammatical, because all 19 merges were resolved correctly.

The three sentences flagged only at the end are the ones the live layer had no chance of catching, and all three turn out to be the same problem. They are the one-word fragments `cu`, `ft` and `ham`, left behind when the sentence splitter cut on an abbreviation. Each one is a legal token that no check complains about while it is typed, and each one scores around -10 per word at the end because a one word sentence of a rare token has nothing to average against. The verdict is right that something is wrong and wrong about what it is.

That is the general weakness of using a language model score as a grammaticality verdict: it measures familiarity, not well-formedness. Sentence 5 of run 2, "pedro spent a whole winter very unhappily", is a perfectly good sentence that scores -9.90 per word and is called ungrammatical, because `pedro` and `unhappily` are close to absent from Brown.

5.2 PCFG against the n-gram models

The two kinds of score catch different things and it shows in the numbers. Only 16.4 percent of the live sentences parsed, against 38.5 percent of treebank sentences of comparable length. Passage sentences are longer, come from Gutenberg and Brown rather than the Wall Street Journal, and carry words the pruned grammar never saw.

Where the parser does fire it is the only score that reacts to structure. "the quick brown fox jumps over the lazy dog" parses at -4.72 per word and is chosen over both n-gram scores, which rate the same sentence at -8.30 and -8.67 because the individual word pairs are uncommon in Brown. That is the split in a nutshell: the parser is asking whether the words form a sentence, and the n-grams are asking whether these particular words tend to follow one another.

The n-grams catch the local damage the parser cannot see. Every real-word suggestion in both sample runs came from the bigram check, and every one of them is a local word pair decision: `so -> do`, `hans -> has`, `falls -> calls`, `met -> let`. The parser has no opinion on any of those, because the substitutions are all the same part of speech and leave the tree intact.

The two n-gram models also disagree by domain, which is worth reporting. On the Brown passage in run 1 the trigram beats the bigram on every sentence, around -3.6 per word against -5.4. On the Gutenberg passage in run 2 the ordering flips and the bigram is better on five of six sentences. Add-k gives no real backoff, so out of domain the trigram's sparse contexts hurt it more than they help. This is why the decision rule checks trigram coverage before trusting it.

5.3 Trigger interval and merge probability against false alerts

Section 1.1 and 1.2 give the measurements. In short, p controls how much genuine work the segmentation layer has, and it does not affect the false-alert floor of 1.3 alerts per 100 words, which is set by out-of-vocabulary words rather than by merging. N controls the grammar false-

alert rate, which is 12 percent at N = 10 and rises in both directions, and it also bounds how late an error can be caught. Since the check costs 0.5 ms, the choice of N is driven entirely by false alerts and not at all by cost.

5.4 Interactions between the sub-systems

The sub-systems feed each other, and the errors compound in both directions.

A wrong split creates work for the speller, and the speller then makes it worse. Run 3 in `reports/sample_run_seed7.txt` has two clean examples. `fastidiousness` is not in Q1's vocabulary, so the decoder split it into `fastidious` and `ness`, and `ness` is not a word either, so the spelling check turned it into `less`. `lamented` went the same way into `la` and `mented`, the speller made `mented` into `melted`, and the next grammar trigger then suggested `la -> a` on top of that. One segmentation mistake travelled through all three sub-systems and came out as three wrong words. The order of the checks makes this unavoidable, since the spelling check is defined to run on whatever the segmentation check leaves behind.

Out-of-vocabulary proper nouns are the same story. Run 3 split `Musgrove` into `mus` and `grove` and then corrected `mus` to `must`, and run 2 turned the name `piedro` into `pedro` twice. A corrector whose only notion of correctness is corpus frequency has nothing sensible to say about a name.

A split can also change which method the decision rule picks. In the seed 9 passage the single word `Highlander` was split into `high` and `lander`, and `lander` is not in the trigram vocabulary. Coverage for that sentence dropped to 78 percent, below the 80 percent threshold, so "murder is a sin said the immovable high lander" was scored by the bigram at -6.34 rather than by the trigram at -8.20. A segmentation decision about one word decided which language model judged the sentence.

Whether a repair flips the parse itself is a fair question and the answer is mostly no. Parsing each repaired sentence and its untouched original, over the 60 sentences of the sweep that carried a repair and stayed inside the 25 word cap in both versions, 53 parsed or failed identically, 4 parsed only after the repair and 3 only before it. The fox sentence from the screenshot is typical: typed as `jumpsover` it already parses at -4.97 per word and the split only improves it to -4.72. The parser's unknown word rule is why. An unseen token falls back to its predicted POS tag, so a merged token enters the chart as an ordinary noun and the tree closes over it regardless.

The seven sentences that do flip go both ways in almost equal numbers, which says the effect is noise rather than improvement. In the seed 9 passage `asperity` was wrongly split into `as` and `purity`, and "said macian with a sudden as purity" parses while the correct sentence does not. Two sentences in the seed 14 passage went the other way and stopped parsing once their merges were resolved. Adding a word changes the span structure, and with a grammar this sparse that is close to a coin toss.

6. Sample runs

Run 1, Brown passage, seed 1

Full transcript in `reports/sample_run_seed1.txt`. 144 tokens, 13 segmentation alerts, 0 spelling alerts, 5 grammar alerts.

#	sentence	pcfg	bigram	trigram	chosen	verdict	merges	fixes
1	the brothers continued to help each other ...	unparseable	-5.42	-3.69	trigram	grammatical	4	0
2	they supplemented their income by small gov...	unparseable	-6.05	-3.59	trigram	grammatical	1	0
3	so impressive were those serious years of s...	unparseable	-5.43	-3.73	trigram	grammatical	3	0

#	sentence	pcfg	bigram	trigram	chosen	verdict	merges	fixes
4	there is a struggle there in which if he fa...	unparseable	-5.34	-3.54	trigram	grammatical	3	0
5	i lived in this on ward driving contest whe...	unparseable	-6.45	-5.09	trigram	grammatical	6	0
6	he openly proclaimed his pleasure in lectur...	unparseable	-6.31	-3.59	trigram	grammatical	2	0

Thirteen segmentation alerts recovered nineteen word boundaries and twelve of the thirteen splits were right. The exception is in sentence 5, where `onward-drivingcontestwhere` became `on ward driving contest where`, and the trigram score of -5.09 is visibly worse than the other five sentences as a result. All five grammar alerts in this run were real-word suggestions and all five were wrong, which is the cost of the 2.0 nat margin inherited from Q3.

Run 2, Gutenberg passage, seed 5

Full transcript in `reports/sample_run_seed5.txt`. 144 tokens, 11 segmentation alerts, 4 spelling alerts, 5 grammar alerts.

#	sentence	pcfg	bigram	trigram	chosen	verdict	merges	fixes
1	pedro was taunted and treated with contempt...	unparseable	-6.85	-7.04	trigram	grammatical	1	1
2	his father when he found that his son s sma...	unparseable	-5.70	-7.56	trigram	grammatical	2	0
3	all the food or clothes that he had at home...	unparseable	-6.14	-6.46	trigram	grammatical	5	0
4	you must eat this now instead of gourd and ...	unparseable	-8.02	-8.76	trigram	borderline	1	2
5	pedro spent a whole winter very unhappily	unparseable	-9.03	-9.90	trigram	ungrammatical	1	1
6	he expected that all his old tricks and esp...	unparseable	-6.50	-8.25	trigram	grammatical	1	0

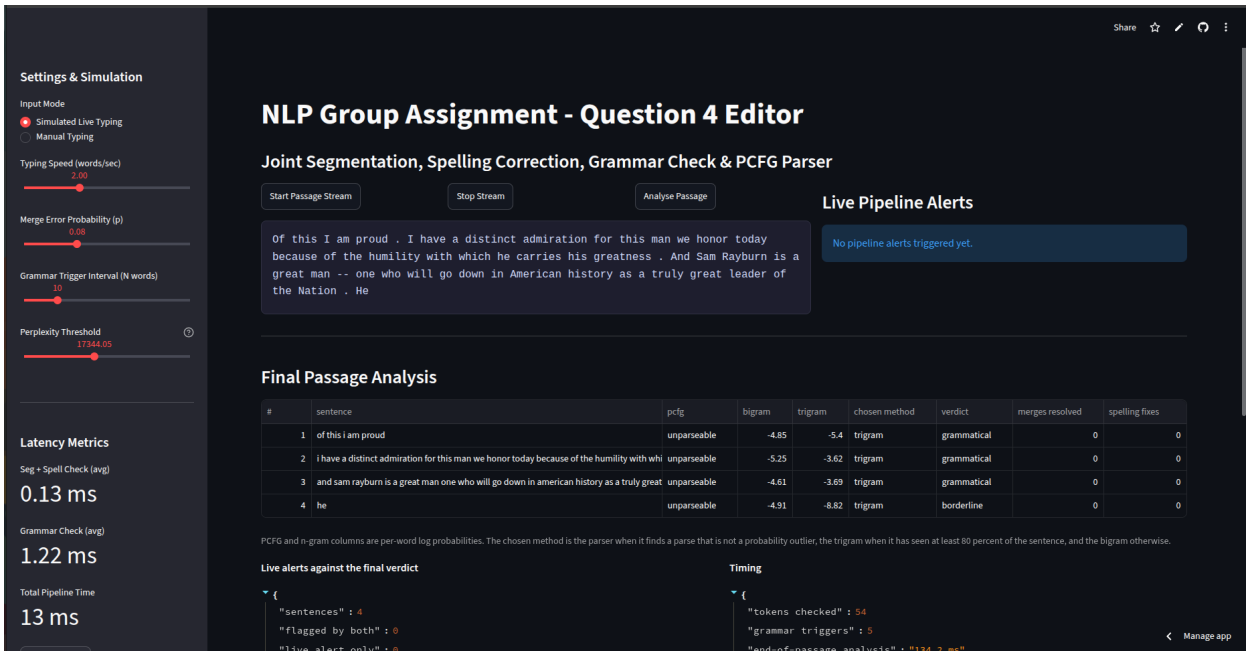
The contrast with run 1 is the point. Same settings and the same token count, but the text is out of domain, the bigram beats the trigram on five of six sentences, and the two sentences that carry spelling corrections of proper nouns and rare words are the two that fail the verdict. This run also has the only perplexity-driven alert of the three: the final trigger fired at 32,426 against the threshold of 17,344, with no real-word suggestion attached to it.

Run 3, Gutenberg passage, seed 7

Full transcript in `reports/sample_run_seed7.txt`. This is the longest of the three at 364 tokens, and it is where the interaction examples in section 5.4 come from. All six sentences were judged grammatical and 29 merges were resolved, but three of the segmentation splits were wrong in a way that cascaded: `fastidiousness`, `lamented` and `Musgrove` are all real words that the decoder broke apart because Q1 had never seen them.

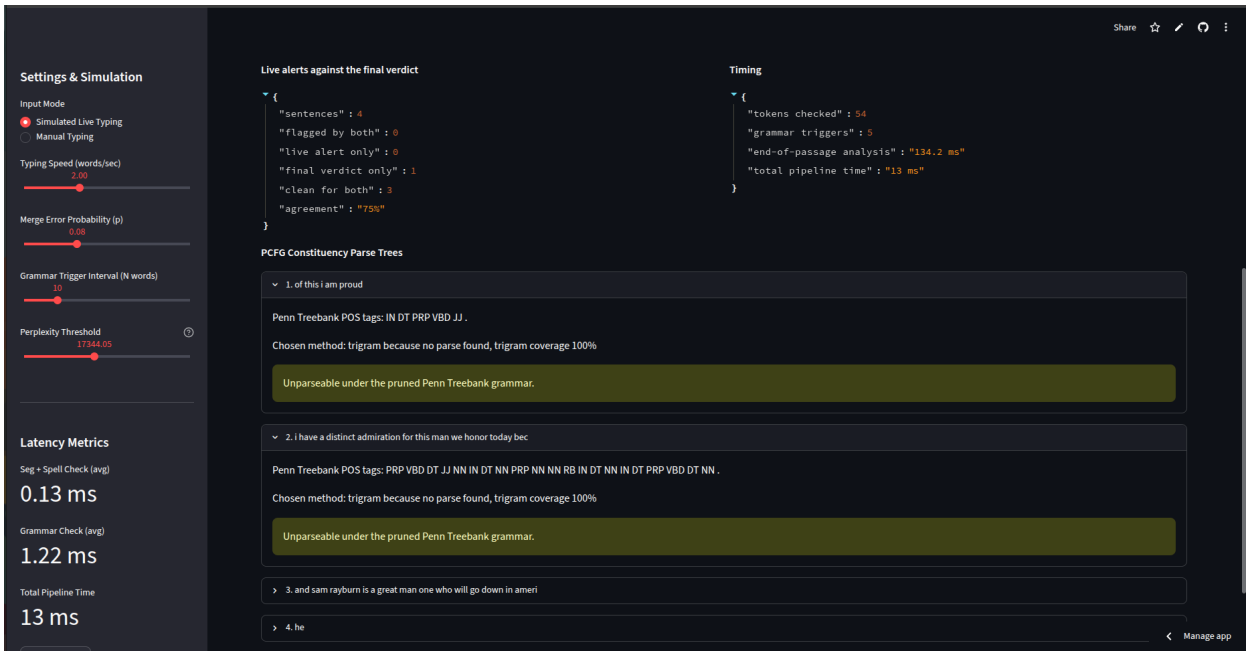
7. Live deployment

The Streamlit app is in `app.py` and runs with `venv/bin/streamlit run app.py`. It has two modes. Simulated live typing streams a sampled passage token by token with a configurable delay, and manual typing processes each new word as it is entered rather than waiting for the whole passage. Both modes share `q4/pipeline.py` with the scripts, so the alerts in the browser are produced by the same code as the transcripts above.



Live alerts in the editor

The first screenshot is Simulated Live Typing mode on a Brown passage about Sam Rayburn, after the stream finished and Analyse Passage ran. The Final Passage Analysis table shows all four sentences: the PCFG fails to parse any of them under the pruned grammar, so the method-selection rule (§3) falls back to the trigram for every row, which has full vocabulary coverage on this passage. Three sentences come back grammatical and the one-word sentence 4 ("he") is borderline. No merges or spelling fixes landed in this run.



End of passage analysis

The second screenshot is the same session scrolled further down. "Live alerts against the final verdict" (§5.1's live metric) shows 3 of the 4 sentences clean under both the live per-token checks and the final verdict, with 1 sentence flagged by the final verdict only — a 75% agreement rate. Timing shows 54 tokens streamed, 5 grammar triggers fired, a 134.2 ms end-of-passage analysis,

and 13 ms of total live pipeline time. The sidebar's latency metrics (0.13 ms average segmentation+spelling check, 1.22 ms average grammar check) match the numbers in §4. Below that, the PCFG Constituency Parse Trees panel shows why the parser lost every sentence in this run: sentence 1 ("of this i am proud") is tagged IN DT PRP VBD JJ . and comes back unparseable under the pruned grammar, and sentence 2 is unparseable for the same reason.

8. Limitations

- The pruned grammar is the main limit on the parser. Keeping every production instead of only those seen twice raises the parse rate from 41 percent to 65 percent on a sample of 150 treebank sentences, but the grammar pickle grows to 116 MB, which is over GitHub's file limit, and parsing slows from 34 ms to 1,048 ms per sentence. At a second per sentence the end-of-passage analysis would stop being interactive, so the pruned grammar stays and the decision rule is built to cope with unparseable sentences instead.
- Very short fragments score well and are called grammatical. A one word sentence such as `mccormack` has almost nothing to average over, so its per-word score is dominated by the sentence-end probability and lands above the floor.
- The first parse in a process pays about 200 ms to build the grammar's rule index, and a cold end-of-passage analysis comes in around 700 ms as a result. Every analysis after that in the same session runs in 15 to 50 ms.
- Add-k is a weak smoother. It is what the brief asks for, and the perplexities in section 1.3 show what it costs compared to the Witten-Bell model Q1 uses for the same corpus.
- The real-word check produces most of the grammar alerts and most of them are wrong. The 2.0 nat margin comes from Q3 and was tuned for sentence-level correction, not for ten-word windows that can cut across a sentence boundary.
- Sentences are cut on final punctuation, with a guard for single letter initials. An abbreviation such as `U.S.` still splits a sentence in two.